

EVENT MONITORING SYSTEM TECHNICAL PROJECT DOCUMENTATION

Generated from the repository on 2026-09-22. This document describes the implemented Kubernetes-focused microservices system, its code, infrastructure, operations, and current validation status.

1. EXECUTIVE SUMMARY

The Event Monitoring System receives application, security, and performance events through a Flask ingestion API. Events are cached in Redis, published durably to RabbitMQ, processed asynchronously by a FastAPI worker, persisted to PostgreSQL, and exposed through a query API and Flask dashboard. High and critical events are routed to a notification worker. Prometheus metrics support operational monitoring and Grafana visualization.

2. ARCHITECTURE

The runtime flow is: client or test script -> event-ingestor -> RabbitMQ event_queue -> event-processor -> PostgreSQL processed_events. The processor also writes a short-lived Redis record and publishes high-severity events to notification_queue. The dashboard calls event-query-api, which reads PostgreSQL. Kubernetes supplies service discovery, health probes, resource limits, and workload orchestration.

CORE COMPONENTS

- event-ingestor: Flask POST /api/events endpoint, validation, Redis cache, RabbitMQ publication, metrics, and critical-event persistence.
- event-processor: FastAPI health/metrics surface and a background RabbitMQ consumer with event-specific processing rules.
- event-query-api: FastAPI read API for recent processed events and severity/status statistics.
- notification-service: RabbitMQ worker that simulates notifications for high-severity events and exposes Prometheus metrics on port 9000.
- dashboard: Flask HTML dashboard refreshed every ten seconds through the query API.
- PostgreSQL, RabbitMQ, Redis: stateful and messaging dependencies deployed in Kubernetes.

3. EVENT CONTRACT

A valid event must contain event_type, source, and severity. message is optional, while metrics and metadata may carry structured values. The ingestor adds timestamp, ingestion_time, and status=pending. The processor adds processed_at and status=processed, or records an error and reports failure.

```
POST /api/events Content-Type: application/json {  "event_type": "security_alert",
"source": "auth-service",  "severity": "critical",  "message": "Multiple failed login
attempts",  "metadata": {"attempts": 12} }
```

4. SERVICE APIS

EVENT-INGESTOR

- GET /health returns service health.
- GET /metrics returns Prometheus exposition data.
- POST /api/events validates and accepts an event, returning HTTP 202.

EVENT-QUERY-API

- GET /health returns service health.
- GET /events?limit=25 returns the newest processed events; limit is clamped to 1..100.

- GET /events/stats returns grouped severity and status counts.
- GET /metrics returns Prometheus exposition data.

EVENT-PROCESSOR

- GET /health and GET /metrics provide probe and monitoring surfaces.

DASHBOARD

- GET /health returns service health.
- GET / renders severity cards and recent processed events.

5. PROCESSING RULES

- security_alert events with critical or high severity generate notification messages.
- performance_metric events with cpu_percent above 90 receive alert=High CPU Usage.
- application_log messages containing ERROR or Exception receive needs_attention=true.
- Successful queue messages are acknowledged. Processing failures are now negatively acknowledged and requeued, preventing silent loss.

6. PERSISTENCE AND DATA MODEL

The database initialization Job creates events and processed_events tables. events stores critical/high events accepted synchronously by the ingestor. processed_events stores the processor output used by the query API and dashboard. JSONB metadata preserves the full event payload and derived fields. Timestamp and severity/status indexes support the primary access patterns.

7. KUBERNETES DEPLOYMENT

The manifests define the event-monitoring namespace, Deployments for application workers, StatefulSets for PostgreSQL/RabbitMQ/Redis, Services for internal discovery and selected NodePort access, ConfigMaps, Secrets, probes, resource requests/limits, Ingress, and a database initialization Job. The setup flow builds and pushes images, updates image registry references, applies the namespace and workloads, waits for readiness, and supports port-forwarding for local access.

```
kubectll get pods -n event-monitoring kubectll port-forward -n event-monitoring svc/event-ingestor-service 8081:80 bash scripts/test-events.sh kubectll port-forward -n event-monitoring svc/dashboard-service 8080:80
```

8. TERRAFORM AWS PRODUCTION DIRECTION

The Terraform area models a production-oriented AWS path with VPC, EKS, RDS PostgreSQL, and Redis modules. This is distinct from the local Kubernetes manifests: the manifests run the playground stack, while Terraform provisions cloud infrastructure intended to host or support a production deployment. Review variables, credentials, state management, networking, and secret handling before applying to an AWS account.

9. CI/CD AND OBSERVABILITY

The Jenkins pipeline builds service images, pushes them to a registry, deploys Kubernetes resources, and includes operational stages for verification and rollback. Prometheus configuration scrapes application metrics, while Grafana dashboard JSON provides visualization. Metric families include events_received_total, events_processed_total, and notifications_sent_total.

10. SECURITY AND RELIABILITY NOTES

- Default credentials exist in application fallbacks and development manifests; production

deployments must use managed secrets and rotate credentials.

- Database connections are opened per operation; production hardening should add pooling, timeouts, and retry/backoff policies.
- Queue messages are durable and consumers use prefetch_count=1. Requeue behavior protects against transient processing failures, but a dead-letter strategy should be added for poison messages.
- The dashboard uses server-side template rendering and escapes values through Jinja; production deployments should add authentication, TLS, and access controls.
- Automated unit/integration tests are not present in the repository. The current verified check is Python compilation; full readiness still requires container, Kubernetes, database, queue, and end-to-end smoke testing.

11. SOURCE CODE APPENDIX

The following appendix includes the implemented application source files. Infrastructure manifests, Terraform modules, Dockerfiles, requirements files, and shell scripts remain in their repository paths and are listed in the project inventory.

APPENDIX: EVENT-INGESTOR (APPLICATIONS/EVENT-INGESTOR/APP.PY)

```
from flask import Flask, request, jsonify
import redis
import psycopg2
import json
import os
from datetime import datetime
import pika
from prometheus_client import Counter, generate_latest, CONTENT_TYPE_LATEST

app = Flask(__name__)

# Configuration
REDIS_HOST = os.getenv('REDIS_HOST', 'localhost')
REDIS_PORT = int(os.getenv('REDIS_PORT', 6379))
DB_HOST = os.getenv('DB_HOST', 'localhost')
DB_NAME = os.getenv('DB_NAME', 'eventdb')
DB_USER = os.getenv('DB_USER', 'postgres')
DB_PASSWORD = os.getenv('DB_PASSWORD', 'password')
RABBITMQ_HOST = os.getenv('RABBITMQ_HOST', 'localhost')
RABBITMQ_PORT = int(os.getenv('RABBITMQ_PORT', 5672))
RABBITMQ_USER = os.getenv('RABBITMQ_USER', 'admin')
RABBITMQ_PASSWORD = os.getenv('RABBITMQ_PASSWORD', 'admin')

EVENTS_RECEIVED = Counter('events_received_total', 'Total events received by the ingestor', ['severity', 'event_type'])

# Initialize connections
redis_client = redis.Redis(host=REDIS_HOST, port=REDIS_PORT, decode_responses=True)
def get_db_connection():
    return psycopg2.connect(host=DB_HOST, database=DB_NAME, user=DB_USER, password=DB_PASSWORD)

def publish_to_queue(event_data):
    """Publish event to RabbitMQ for async processing"""
    credentials = pika.PlainCredentials(RABBITMQ_USER, RABBITMQ_PASSWORD)
    connection = pika.BlockingConnection(pika.ConnectionParameters(host=RABBITMQ_HOST, port=RABBITMQ_PORT, credentials=credentials))
    channel = connection.channel()
    channel.queue_declare(queue='event_queue', durable=True)
    channel.basic_publish(exchange='', routing_key='event_queue', body=json.dumps(event_data), properties=pika.BasicProperties(delivery_mode=2, # make message persistent))
    connection.close()

@app.route('/health', methods=['GET'])
def health_check():
    return jsonify({"status": "healthy", "service": "event-ingestor"}), 200

@app.route('/metrics', methods=['GET'])
def metrics():
    return generate_latest(), 200, {'Content-Type': CONTENT_TYPE_LATEST}

@app.route('/api/events', methods=['POST'])
def receive_event():
    try:
        event_data = request.json
    except:
        return jsonify({"error": "Request body must be JSON object"}), 400

    # Validate required fields
    required_fields = ['event_type', 'source', 'severity']
    for field in required_fields:
        if field not in event_data:
            return jsonify({"error": f"Missing field: {field}"}), 400

    # Add metadata
    event_data['timestamp'] = datetime.utcnow().isoformat()
    event_data['ingestion_time'] = datetime.utcnow().isoformat()
    event_data['status'] = 'pending'
```

```

Store in Redis for real-time access      event_id =
f"event:{datetime.utcnow().timestamp()}"      redis_client.hset(event_id,
mapping=string_mapping(event_data))      redis_client.expire(event_id, 3600) # Expire
after 1 hour      # Publish to queue for async processing
publish_to_queue(event_data)      EVENTS_RECEIVED.labels(
severity=event_data['severity'],      event_type=event_data['event_type']
).inc()      # Store in PostgreSQL (sync for critical events)      if
event_data['severity'] in ['critical', 'high']:      store_in_database(event_data)
return jsonify({      "message": "Event received successfully",
"event_id": event_id,      "timestamp": event_data['timestamp']      }), 202
except Exception as e:      app.logger.error(f"Error processing event: {str(e)}")
return jsonify({"error": "Internal server error"}), 500 def store_in_database(event_data):
"""Store event in PostgreSQL"""      conn = get_db_connection()      cur =
conn.cursor()      cur.execute("""      INSERT INTO events (event_type,
source, severity, message, metadata, timestamp)      VALUES (%s, %s, %s, %s, %s, %s)
""", (      event_data.get('event_type'),      event_data.get('source'),
event_data.get('severity'),      event_data.get('message', ''),
json.dumps(event_data.get('metadata', {})),      event_data.get('timestamp')
))      conn.commit()      cur.close()      conn.close()
except Exception as e:      app.logger.error(f"Database error: {str(e)}") def
string_mapping(event_data):      """Redis hashes need scalar values; nested fields are stored
as JSON."""      return {      key: value if isinstance(value, str) else json.dumps(value)
for key, value in event_data.items()      } if __name__ == '__main__':
app.run(host='0.0.0.0', port=5000, debug=False)

```

APPENDIX: EVENT-PROCESSOR (APPLICATIONS/EVENT-PROCESSOR/MAIN.PY)

```

import json import logging import os import threading from datetime import datetime from
typing import Any, Dict import pika import psycopg2 import redis from fastapi import
FastAPI from prometheus_client import CONTENT_TYPE_LATEST, Counter, generate_latest from
starlette.responses import Response app = FastAPI(title="Event Processor") REDIS_HOST =
os.getenv("REDIS_HOST", "localhost") REDIS_PORT = int(os.getenv("REDIS_PORT", "6379"))
DB_HOST = os.getenv("DB_HOST", "localhost") DB_NAME = os.getenv("DB_NAME", "eventdb")
DB_USER = os.getenv("DB_USER", "postgres") DB_PASSWORD = os.getenv("DB_PASSWORD",
"password") RABBITMQ_HOST = os.getenv("RABBITMQ_HOST", "localhost") RABBITMQ_PORT =
int(os.getenv("RABBITMQ_PORT", "5672")) RABBITMQ_USER = os.getenv("RABBITMQ_USER", "admin")
RABBITMQ_PASSWORD = os.getenv("RABBITMQ_PASSWORD", "admin")
logging.basicConfig(level=os.getenv("LOG_LEVEL", "INFO")) logger =
logging.getLogger(__name__) EVENTS_PROCESSED = Counter(      "events_processed_total",
"Total events processed by the processor",      ["severity", "event_type", "status"], )
class EventProcessor:      def __init__(self) -> None:      self.redis_client =
redis.Redis(      host=REDIS_HOST,      port=REDIS_PORT,
decode_responses=True,      )      def get_db_connection(self):      return
psycopg2.connect(      host=DB_HOST,      database=DB_NAME,
user=DB_USER,      password=DB_PASSWORD,      )      def process_event(self,
event_data: Dict[str, Any]) -> bool:      status = "processed"      try:
event_data["processed_at"] = datetime.utcnow().isoformat()      event_data["status"]
= status      if event_data.get("event_type") == "security_alert":
self.handle_security_alert(event_data)      elif event_data.get("event_type") ==
"performance_metric":      self.handle_performance_metric(event_data)
elif event_data.get("event_type") == "application_log":
self.handle_application_log(event_data)      self.store_event(event_data)

```

```

event_id = f"processed:{event_data.get('timestamp', event_data['processed_at'])}"
self.redis_client.hset(event_id, mapping=self._string_mapping(event_data))
self.redis_client.expire(event_id, 3600) logger.info("Processed event: %s",
event_data.get("event_type")) return True except Exception as exc:
status = "failed" event_data["status"] = status event_data["error"]
= str(exc) logger.exception("Error processing event") return False
finally:
EVENTS_PROCESSED.labels(
severity=event_data.get("severity", "unknown"),
event_type=event_data.get("event_type", "unknown"), status=status,
).inc()
def handle_security_alert(self, event_data: Dict[str, Any]) -> None: if
event_data.get("severity") in ["critical", "high"]: logger.warning("Security
alert needs notification: %s", event_data)
def handle_performance_metric(self,
event_data: Dict[str, Any]) -> None: metrics = event_data.get("metrics", {})
if metrics.get("cpu_percent", 0) > 90: event_data["alert"] = "High CPU Usage"
def handle_application_log(self, event_data: Dict[str, Any]) -> None: log_message =
event_data.get("message", "") if "ERROR" in log_message or "Exception" in
log_message: event_data["needs_attention"] = True
def store_event(self,
event_data: Dict[str, Any]) -> None: with self.get_db_connection() as conn:
with conn.cursor() as cur: cur.execute(
INSERT INTO processed_events
(event_type, source, severity, message,
metadata, original_timestamp, processed_timestamp, status)
VALUES (%s, %s, %s, %s, %s, %s, %s, %s)
event_data.get("event_type"),
event_data.get("source"),
event_data.get("severity"),
event_data.get("message", ""),
event_data.get("timestamp"),
event_data.get("status"),
"processed"),
def publish_notification(self,
event_data: Dict[str, Any]) -> None:
credentials =
pika.PlainCredentials(RABBITMQ_USER, RABBITMQ_PASSWORD) connection =
pika.BlockingConnection(
pika.ConnectionParameters(
host=RABBITMQ_HOST,
port=RABBITMQ_PORT,
credentials=credentials,
) channel = connection.channel()
channel.queue_declare(queue="notification_queue", durable=True)
channel.basic_publish(
exchange="",
routing_key="notification_queue",
body=json.dumps(event_data),
properties=pika.BasicProperties(delivery_mode=2),
) connection.close()
@staticmethod
def _string_mapping(event_data: Dict[str, Any]) -> Dict[str, str]:
return {
key: value if isinstance(value, str) else json.dumps(value)
}
for key, value in event_data.items(): processor = EventProcessor()
def consume_events() -> None: while True: try:
credentials =
pika.PlainCredentials(RABBITMQ_USER, RABBITMQ_PASSWORD) connection =
pika.BlockingConnection(
pika.ConnectionParameters(
host=RABBITMQ_HOST,
port=RABBITMQ_PORT,
credentials=credentials,
) channel =
connection.channel() channel.queue_declare(queue="event_queue", durable=True)
channel.basic_qos(prefetch_count=1)
def callback(ch, method, properties, body):
try:
event_data = json.loads(body)
processed =
processor.process_event(event_data) if processed and
event_data.get("severity") in ["critical", "high"]:
processor.publish_notification(event_data) if processed:
ch.basic_ack(delivery_tag=method.delivery_tag)
else:
ch.basic_nack(delivery_tag=method.delivery_tag, requeue=True) except

```

```

Exception:                logger.exception("Could not process queue message")
ch.basic_nack(delivery_tag=method.delivery_tag, requeue=True)
channel.basic_consume(queue="event_queue", on_message_callback=callback)
logger.info("Event processor consumer started")                channel.start_consuming()
except Exception:        logger.exception("Event processor consumer disconnected;
retrying")                threading.Event().wait(5)    @app.on_event("startup") async def
startup_event() -> None:    thread = threading.Thread(target=consume_events, daemon=True)
thread.start()    @app.get("/health") async def health_check():    return {"status":
"healthy", "service": "event-processor"}    @app.get("/metrics") async def metrics():
return Response(generate_latest(), media_type=CONTENT_TYPE_LATEST)

```

APPENDIX: EVENT-QUERY-API (APPLICATIONS/EVENT-QUERY-API/MAIN.PY)

```

import os from typing import Any import psycopg2 from fastapi import FastAPI from
prometheus_client import CONTENT_TYPE_LATEST, generate_latest from starlette.responses
import Response app = FastAPI(title="Event Query API") DB_HOST = os.getenv("DB_HOST",
"localhost") DB_NAME = os.getenv("DB_NAME", "eventdb") DB_USER = os.getenv("DB_USER",
"postgres") DB_PASSWORD = os.getenv("DB_PASSWORD", "password") def get_db_connection():
return psycopg2.connect(        host=DB_HOST,        database=DB_NAME,
user=DB_USER,        password=DB_PASSWORD,    ) def rows_to_dicts(cursor, rows) ->
list[dict[str, Any]]:    columns = [column.name for column in cursor.description]
return [dict(zip(columns, row)) for row in rows] @app.get("/health") async def
health_check():    return {"status": "healthy", "service": "event-query-api"}
@app.get("/events") async def recent_events(limit: int = 25):    limit = max(1, min(limit,
100))    with get_db_connection() as conn:        with conn.cursor() as cur:
cur.execute(        """        SELECT id, event_type, source, severity,
message, status,        original_timestamp, processed_timestamp
FROM processed_events        ORDER BY processed_timestamp DESC
LIMIT %s        """,        (limit,),    )    return
{"events": rows_to_dicts(cur, cur.fetchall())} @app.get("/events/stats") async def
event_stats():    with get_db_connection() as conn:        with conn.cursor() as cur:
cur.execute(        """        SELECT severity, COUNT(*) AS count
FROM processed_events        GROUP BY severity        ORDER BY count DESC
""")        severity_counts = rows_to_dicts(cur, cur.fetchall())
cur.execute(        """        SELECT status, COUNT(*) AS count
FROM processed_events        GROUP BY status        ORDER BY count DESC
""")        status_counts = rows_to_dicts(cur, cur.fetchall())    return
{        "severity_counts": severity_counts,        "status_counts": status_counts,    }
@app.get("/metrics") async def metrics():    return Response(generate_latest(),
media_type=CONTENT_TYPE_LATEST)

```

APPENDIX: NOTIFICATION-SERVICE (APPLICATIONS/NOTIFICATION-SERVICE/MAIN.PY)

```

import json import logging import os import time import pika from prometheus_client import
Counter, start_http_server RABBITMQ_HOST = os.getenv("RABBITMQ_HOST", "localhost")
RABBITMQ_PORT = int(os.getenv("RABBITMQ_PORT", "5672")) RABBITMQ_USER =
os.getenv("RABBITMQ_USER", "admin") RABBITMQ_PASSWORD = os.getenv("RABBITMQ_PASSWORD",
"admin") logging.basicConfig(level=os.getenv("LOG_LEVEL", "INFO")) logger =
logging.getLogger(__name__) NOTIFICATIONS_SENT = Counter(    "notifications_sent_total",
"Total simulated notifications sent",    ["severity", "event_type"], ) def connect():
credentials = pika.PlainCredentials(RABBITMQ_USER, RABBITMQ_PASSWORD)    return
pika.BlockingConnection(    pika.ConnectionParameters(        host=RABBITMQ_HOST,

```

```

port=RABBITMQ_PORT, credentials=credentials, ) ) def main():
start_http_server(9000) while True: try: connection = connect()
channel = connection.channel() channel.queue_declare(queue="notification_queue",
durable=True) def callback(ch, method, properties, body): event
= json.loads(body) logger.warning("Simulated alert
notification: severity=%s type=%s source=%s", event.get("severity"),
event.get("event_type"), event.get("source"), )
NOTIFICATIONS_SENT.labels(severity=event.get("severity", "unknown"),
event_type=event.get("event_type", "unknown"), ).inc()
ch.basic_ack(delivery_tag=method.delivery_tag)
channel.basic_qos(prefetch_count=1) channel.basic_consume(
queue="notification_queue", on_message_callback=callback, )
logger.info("Notification service started") channel.start_consuming()
except Exception: logger.exception("Notification service disconnected;
retrying") time.sleep(5) if __name__ == "__main__": main()

```

APPENDIX: DASHBOARD (APPLICATIONS/DASHBOARD/APP.PY)

```

import os import requests from flask import Flask, jsonify, render_template_string app =
Flask(__name__) QUERY_API_URL = os.getenv("QUERY_API_URL", "http://event-query-api-
service") HTML = """ <!doctype html> <html lang="en"> <head> <meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1"> <meta http-
equiv="refresh" content="10"> <title>Event Monitoring Dashboard</title> <style> body
{ margin: 0; font-family: Arial, sans-serif; background: #0f172a; color: #e5e7eb; }
header { padding: 24px 32px; background: #111827; border-bottom: 1px solid #334155; }
main { padding: 24px 32px; display: grid; gap: 24px; } h1 { margin: 0; font-size: 28px;
} h2 { margin: 0 0 12px; font-size: 18px; } table { width: 100%; border-collapse:
collapse; background: #111827; } th, td { padding: 12px; border-bottom: 1px solid
#334155; text-align: left; } th { color: #93c5fd; } .grid { display: grid; grid-
template-columns: repeat(auto-fit, minmax(180px, 1fr)); gap: 16px; } .card { background:
#111827; border: 1px solid #334155; border-radius: 8px; padding: 16px; } .count { font-
size: 32px; font-weight: 700; margin-top: 8px; } .empty { color: #94a3b8; } </style>
</head> <body> <header><h1>Event Monitoring Dashboard</h1></header> <main> <section>
<h2>Severity Counts</h2> <div class="grid"> {% for item in
stats.severity_counts %} <div class="card"> <div>{{ item.severity }}</div>
<div class="count">{{ item.count }}</div> </div> {% else %} <div
class="empty">No processed events yet.</div> {% endfor %} </div>
</section> <section> <h2>Recent Processed Events</h2> <table>
<thead> <tr><th>ID</th><th>Type</th><th>Source</th><th>Severity</th><th>Status</th>
<th>Message</th></tr> </thead> <tbody> {% for event in events %}
<tr> <td>{{ event.id }}</td> <td>{{ event.event_type }}</td>
<td>{{ event.source }}</td> <td>{{ event.severity }}</td> <td>{{
event.status }}</td> <td>{{ event.message }}</td> </tr> {%
else %} <tr><td colspan="6" class="empty">Send a test event to populate the
dashboard.</td></tr> {% endfor %} </tbody> </table> </section>
</main> </body> </html> """ @app.get("/health") def health_check(): return
jsonify({"status": "healthy", "service": "dashboard"}) @app.get("/") def index(): try:
stats = requests.get(f"{QUERY_API_URL}/events/stats", timeout=5).json() events =
requests.get(f"{QUERY_API_URL}/events?limit=25", timeout=5).json() except
requests.RequestException: stats = {"severity_counts": []} events =
{"events": []} return render_template_string(HTML, stats=stats,
events=events.get("events", []), )

```